

```
// =====  
// JAI ROOP TEXTILES — IMS System V6  
// Google Apps Script Backend  
// CODE.GS — Full Production Version  
// =====
```

```
// ---- AUTO-TRIGGER: Status change in Order Book ----
```

```
function onEdit(e) {  
  var sheet = e.source.getActiveSheet();  
  if (sheet.getName() == "Order Book" && e.range.getColumn() == 6) {  
    updateMasterAllocatedAndStock(e.source);  
  }  
}
```

```
// ---- ALLOCATED QTY SYNC: Order Book → Master ----
```

```
function updateMasterAllocatedAndStock(ss) {  
  var orderSheet = ss.getSheetByName("Order Book");  
  var masterSheet = ss.getSheetByName("Master");  
  var orderData = orderSheet.getDataRange().getValues();  
  var masterData = masterSheet.getDataRange().getValues();  
  var allocationMap = {};  
  
  for (var i = 1; i < orderData.length; i++) {  
    var itemCode = orderData[i][3];  
    var qty = parseFloat(orderData[i][4]) || 0;  
    var status = orderData[i][5];
```

```

    if (status == "In Production" || status == "Pending") {
        allocationMap[itemCode] = (allocationMap[itemCode] || 0) + qty;
    }
}

for (var j = 1; j < masterData.length; j++) {
    var masterItemCode = masterData[j][0];
    if (masterItemCode) {
        masterSheet.getRange(j + 1, 9).setValue(allocationMap[masterItemCode] || 0);
    }
}

// =====
// WEB APP ENTRY POINT
// =====

function doGet() {
    return HtmlService.createTemplateFromFile('Index')
        .evaluate()
        .setTitle('IMS Dashboard — Jai Roop Textiles')
        .addMetaTag('viewport', 'width=device-width, initial-scale=1')
        .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}

// =====
// MASTER DATA FETCH

```

```
// =====

function getAllMSData() {

    var ss = SpreadsheetApp.getActiveSpreadsheet();

    var logoBase64 = "";

    try {

        var fileId = "12xaMnUBwVFr_kdKYuJozypKuLWPyHmH";

        var file = DriveApp.getFileById(fileId);

        var blob = file.getBlob();

        logoBase64 = "data:" + blob.getContentType() + ";base64," +
Utilities.base64Encode(blob.getBytes());

    } catch (e) {

        Logger.log("Logo load failed: " + e.toString());

    }

    var masterRaw = getSheetDataFast(ss, "Master");
    var ordersRaw = getSheetDataFast(ss, "Order Book");
    var jobWorkRaw = getSheetDataFast(ss, "Job Work Tracker");
    var dispatchRaw = getSheetDataFast(ss, "Dispatch Log");

    // Unique clients

    var clientSet = {};

    ordersRaw.forEach(function(o) { if (o.ClientName) clientSet[o.ClientName] = true; });

    // Unique workers

    var workerSet = {};

```

```
jobWorkRaw.forEach(function(j) { if (j.WorkerName) workerSet[j.WorkerName] = true; });
```

```
// KPI Analytics (server-side)
```

```
var kpis = computeKPIs(masterRaw, ordersRaw, jobWorkRaw, dispatchRaw);
```

```
return {
```

```
  master: masterRaw,
```

```
  orders: ordersRaw,
```

```
  jobWork: jobWorkRaw,
```

```
  dispatch: dispatchRaw,
```

```
  clients: Object.keys(clientSet).sort(),
```

```
  workers: Object.keys(workerSet).sort(),
```

```
  kpis: kpis,
```

```
  logoUrl: logoBase64,
```

```
  lastSync: Utilities.formatDate(new Date(), Session.getScriptTimeZone(), "dd MMM yyyy,  
hh:mm a")
```

```
};
```

```
}
```

```
// =====
```

```
// KPI COMPUTATION (Server-Side Analytics)
```

```
// =====
```

```
function computeKPIs(master, orders, jobWork, dispatch) {
```

```
  var reorderItems = master.filter(function(i) { return i.ReorderStatus === 'REORDER (LOW)';  
});
```

```
  var activeOrders = orders.filter(function(o) { return o.Status === 'In Production'; });
```

```
  var pendingOrders = orders.filter(function(o) { return o.Status === 'Pending'; });
```

```
var completedOrders = orders.filter(function(o) { return o.Status === 'Completed'; });  
var jwOut = jobWork.filter(function(j) { return j.Status === 'Out'; });
```

```
// Total dispatched volume
```

```
var totalDispatched = dispatch.reduce(function(sum, d) {  
    return sum + (parseFloat(d.DispatchedQty) || 0);  
}, 0);
```

```
// Wastage analysis
```

```
var totalSent = 0, totalWastage = 0;  
jobWork.forEach(function(j) {  
    totalSent += parseFloat(j.SentQtyMtr) || 0;  
    totalWastage += parseFloat(j.WastageMtr) || 0;  
});  
var avgWastagePct = totalSent > 0 ? (totalWastage / totalSent * 100).toFixed(2) : 0;
```

```
// Category breakdown from master
```

```
var categoryMap = {};  
master.forEach(function(item) {  
    var cat = item.Category || 'Other';  
    if (!categoryMap[cat]) categoryMap[cat] = { count: 0, totalStock: 0 };  
    categoryMap[cat].count++;  
    categoryMap[cat].totalStock += parseFloat(item.ClosingBalance) || 0;  
});
```

```
// Top 5 items by closing balance
```

```
var topltems = master

    .filter(function(i) { return (parseFloat(i.ClosingBalance) || 0) > 0; })

    .sort(function(a, b) { return (parseFloat(b.ClosingBalance) || 0) -
(parseFloat(a.ClosingBalance) || 0); })

    .slice(0, 5)

    .map(function(i) { return { code: i.ItemCode, balance: parseFloat(i.ClosingBalance) || 0 };
});
```

```
// Monthly dispatch trend (last 6 months)

var monthlyDispatch = {};

dispatch.forEach(function(d) {

    if (d.DispatchDate) {

        var month = d.DispatchDate.substring(0, 7);

        monthlyDispatch[month] = (monthlyDispatch[month] || 0) +
(parseFloat(d.DispatchedQty) || 0);

    }

});
```

```
// Order fulfilment rate

var totalOrders = orders.length;

var fulfilmentRate = totalOrders > 0 ? ((completedOrders.length / totalOrders) *
100).toFixed(1) : 0;
```

```
return {

    reorderCount: reorderItems.length,

    reorderItems: reorderItems.map(function(i) { return i.ItemCode; }),

    activeOrderCount: activeOrders.length,
```

```

    pendingOrderCount: pendingOrders.length,
    completedOrderCount: completedOrders.length,
    totalOrderCount: totalOrders,
    fulfilmentRate: fulfilmentRate,
    jwLiveCount: jwOut.length,
    totalDispatched: totalDispatched,
    avgWastagePct: avgWastagePct,
    totalItems: master.length,
    categoryBreakdown: categoryMap,
    topItems: topItems,
    monthlyDispatch: monthlyDispatch
};
}

```

```

// =====
// SHEET DATA READER (Fast, Generic)
// =====

function getSheetDataFast(ss, name) {
    var sheet = ss.getSheetByName(name);
    if (!sheet) return [];
    var rows = sheet.getDataRange().getValues();
    if (rows.length < 2) return [];

    var headers = rows.shift().map(function(h) {
        return h.toString().replace(/^[^a-zA-Z0-9]/g, "");
    });
}

```

```

return rows.map(function(row) {
    var obj = {};
    var hasData = false;
    headers.forEach(function(h, i) {
        var val = row[i];
        if (val !== "" && val !== null && val !== undefined) hasData = true;
        if (val instanceof Date) {
            obj[h] = Utilities.formatDate(val, Session.getScriptTimeZone(), "yyyy-MM-dd");
        } else {
            obj[h] = val;
        }
    });
    return hasData ? obj : null;
}).filter(Boolean);
}

```

```
// =====
```

```
// WRITE-BACK: ADD NEW ORDER
```

```
// =====
```

```

function saveNewOrder(orderData) {
    try{
        var ss = SpreadsheetApp.getActiveSpreadsheet();
        var sheet = ss.getSheetByName("Order Book");
        var lastRow = sheet.getLastRow();
    }
}

```

```

// Generate Order ID: ORD-YYYYMMDD-XXX

var today = Utilities.formatDate(new Date(), Session.getScriptTimeZone(), "yyyyMMdd");

var orderId = "ORD-" + today + "-" + String(lastRow).padStart(3, '0');


sheet.appendRow([

    orderId,

    orderData.date || Utilities.formatDate(new Date(), Session.getScriptTimeZone(), "yyyy-MM-dd"),

    orderData.clientName,

    orderData.itemCode,

    parseFloat(orderData.qty),

    orderData.status || "Pending",

    orderData.remarks || ""

]);


// Update allocations

updateMasterAllocatedAndStock(ss);


    return { success: true, orderId: orderId, message: "Order " + orderId + " saved successfully!" };

} catch (e) {

    return { success: false, message: "Error: " + e.toString() };

}

}

// =====

// WRITE-BACK: UPDATE ORDER STATUS

```

```
// =====

function updateOrderStatus(orderId, newStatus) {
  try {
    var ss = SpreadsheetApp.getActiveSpreadsheet();
    var sheet = ss.getSheetByName("Order Book");
    var data = sheet.getDataRange().getValues();

    for (var i = 1; i < data.length; i++) {
      if (data[i][0] == orderId) {
        sheet.getRange(i + 1, 6).setValue(newStatus);
        updateMasterAllocatedAndStock(ss);
        return { success: true, message: "Order " + orderId + " updated to: " + newStatus };
      }
    }
    return { success: false, message: "Order ID not found." };
  } catch (e) {
    return { success: false, message: "Error: " + e.toString() };
  }
}
```

```
// =====
```

```
// WRITE-BACK: ADD NEW JOB WORK ENTRY
```

```
// =====
```

```
function saveNewJobWork(jobData) {
  try {
    var ss = SpreadsheetApp.getActiveSpreadsheet();
```

```

var sheet = ss.getSheetByName("Job Work Tracker");
var lastRow = sheet.getLastRow();

var today = Utilities.formatDate(new Date(), Session.getScriptTimeZone(), "yyyyMMdd");
var jobId = "JOB-" + today + "-" + String(lastRow).padStart(3, '0');

sheet.appendRow([
    jobData.date || Utilities.formatDate(new Date(), Session.getScriptTimeZone(), "yyyy-MM-dd"),
    jobId,
    jobData.orderId,
    jobData.workerName,
    jobData.itemCode,
    parseFloat(jobData.sentQty) || 0,
    parseFloat(jobData.receivedQty) || 0,
    parseFloat(jobData.wastageMtr) || 0,
    jobData.status || "Out",
    parseFloat(jobData.totalPayment) || 0
]);

return { success: true, jobId: jobId, message: "Job Work " + jobId + " saved successfully!"
};
} catch (e) {
    return { success: false, message: "Error: " + e.toString() };
}
}

```

```

// =====

// WRITE-BACK: ADD NEW DISPATCH ENTRY

// =====

function saveNewDispatch(dispatchData) {

    try{

        var ss = SpreadsheetApp.getActiveSpreadsheet();

        var sheet = ss.getSheetByName("Dispatch Log");

        var lastRow = sheet.getLastRow();


        var today = Utilities.formatDate(new Date(), Session.getScriptTimeZone(), "yyyyMMdd");

        var dispId = "DISP-" + today + "-" + String(lastRow).padStart(3, '0');


        sheet.appendRow([

            dispId,

            dispatchData.orderId,

            dispatchData.clientName,

            dispatchData.itemCode,

            parseFloat(dispatchData.dispatchedQty) || 0,

            dispatchData.dispatchDate || Utilities.formatDate(new Date(),

Session.getScriptTimeZone(), "yyyy-MM-dd"),

            dispatchData.challanNo || ""

        ]);


        // Update order status to Completed if flagged

        if (dispatchData.markCompleted) {

            updateOrderStatus(dispatchData.orderId, "Completed");

```

```

    }

    return { success: true, dispId: dispId, message: "Dispatch " + dispId + " logged
successfully!" };

} catch (e) {

    return { success: false, message: "Error: " + e.toString() };

}

}

// =====
// STOCK ALERTS (for notification panel)
// =====

function getStockAlerts() {

    var ss = SpreadsheetApp.getActiveSpreadsheet();
    var master = getSheetDataFast(ss, "Master");
    var alerts = [];

    master.forEach(function(item) {

        var balance = parseFloat(item.ClosingBalance) || 0;
        var allocated = parseFloat(item.AllocatedtoOrders) || 0;

        if (item.ReorderStatus === 'REORDER (LOW)') {

            alerts.push({

                type: 'critical',

                itemCode: item.ItemCode,

                itemName: item.ItemName,

```

```
        balance: balance,
        message: 'CRITICAL: Stock below reorder level!'
    });
} else if (allocated > balance * 0.8) {
    alerts.push({
        type: 'warning',
        itemCode: item.ItemCode,
        itemName: item.ItemName,
        balance: balance,
        message: 'WARNING: Over 80% stock allocated to orders'
    });
}
});

return alerts;
}
```